

Clase 055 — PKI, certificados X.509 y autoridades de certificación

Parte: 2 — **Criptografía aplicada** · Fuente: *Real-World Cryptography* (Wong) e IETF RFC 5280 🕒
Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Comprender cómo se resuelve el problema de "¿de quién es esta clave pública?" mediante la infraestructura de clave pública (PKI): certificados X.509, autoridades de certificación (CA), cadenas de confianza, y mecanismos de revocación (CRL, OCSP). El alumno construirá su propia CA de laboratorio con OpenSSL, emitirá certificados y comprenderá cómo el navegador valida una cadena hasta una raíz de confianza.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Describir** la estructura de un certificado X.509 y sus campos clave.
2. **Explicar** la cadena de confianza (raíz → intermedia → hoja) y su validación.
3. **Crear** una CA propia, emitir un CSR y firmar un certificado con OpenSSL.
4. **Comparar** mecanismos de revocación (CRL, OCSP, OCSP stapling).
5. **Identificar** riesgos de PKI (CA comprometida, misemisión) y mitigaciones (CT, pinning).

📖 Temas

#	Tema	Por qué importa
1	Problema de confianza en claves	Motiva la PKI
2	Certificado X.509	Vincula identidad y clave
3	CA y cadena de confianza	Delegación de confianza
4	CSR y emisión	Cómo se pide un certificado
5	Revocación (CRL/OCSP)	Invalidar antes de expirar
6	Certificate Transparency	Detectar misemisión
7	Let's Encrypt / ACME	PKI moderna automatizada

📖 Definiciones y características

- **PKI**: conjunto de roles, políticas y procedimientos para gestionar certificados de clave pública.
Característica: escala la confianza mediante CAs.

- **Certificado X.509**: documento firmado por una CA que vincula una identidad (dominio) con una clave pública, con validez y extensiones (SAN, uso de clave).
- **Autoridad de certificación (CA)**: entidad de confianza que firma certificados. Su clave raíz está preinstalada en sistemas y navegadores.
- **Cadena de confianza**: raíz → intermedia(s) → hoja; se valida cada firma hasta un ancla de confianza.
- **CSR (Certificate Signing Request)**: solicitud que contiene la clave pública y datos del solicitante; la CA la firma.
- **CRL / OCSP**: listas de revocación y consulta en línea del estado de un certificado. **OCSP stapling** adjunta la respuesta al handshake.
- **Certificate Transparency (CT)**: registros públicos y auditables de certificados emitidos, para detectar emisiones fraudulentas.

Herramientas y preparación

```
openssl version
```

Toda la PKI es de laboratorio: son certificados propios que no serán confiados por navegadores reales, y así debe ser.

Laboratorio guiado

1. Crea tu CA raíz:

```
bash openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out ca.key openssl req -x509 -new -key ca.key -sha256 -days 3650 \ -subj "/CN=Lab Root CA" -out ca.crt
```

2. Genera clave y CSR del servidor:

```
bash openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out srv.key openssl req -new -key srv.key -subj "/CN=lab.local" -out srv.csr
```

3. Firma el certificado del servidor con la CA (con SAN):

```
bash openssl x509 -req -in srv.csr -CA ca.crt -CAkey ca.key -CAcreateserial \ -days 365 -sha256 -extfile <(printf "subjectAltName=DNS:lab.local") -out srv.crt
```

4. Inspecciona y valida la cadena:

```
bash openssl x509 -in srv.crt -text -noout | head -n 20 openssl verify -CAfile ca.crt srv.crt
```

5. **Explora Certificate Transparency**. Consulta un registro público (por ejemplo crt.sh en el navegador) para un dominio y observa cuántos certificados se han emitido; discute cómo esto detecta misemisiones.

Ejercicios

1. Enumera y explica cinco campos de un certificado X.509 real (`openssl x509 -text`).
2. Emite un certificado con varias entradas SAN y verifícalo.
3. Compara CRL, OCSP y OCSP stapling en latencia y privacidad.
4. Explica qué ocurre si se compromete la clave de una CA raíz.

- Investiga el incidente de DigiNotar y sus consecuencias.
- Describe el flujo ACME de Let's Encrypt paso a paso.

Reto verificable

Construye una PKI de dos niveles (raíz + intermedia) y emite un certificado de servidor firmado por la intermedia; configura un servidor TLS de laboratorio que presente la cadena completa. **Criterio de aceptación:** `openssl verify -CAfile ca.crt -untrusted intermedia.crt srv.crt` devuelve OK y un cliente valida la cadena hasta la raíz.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>unable to get local issuer certificate</code>	Falta la intermedia; incluye la cadena completa
Navegador rechaza el certificado	CN sin SAN; los navegadores exigen SAN
<code>certificate has expired</code>	Validez vencida; reemite
Confiar en la raíz de laboratorio en producción	Nunca; usa CAs públicas reales
No revocar tras compromiso de clave	Publica CRL/OCSP y reemite

Preguntas frecuentes

 **¿Por qué hay CAs intermedias?** Para proteger la raíz (offline) y limitar el daño si se compromete una intermedia, que puede revocarse.

 **¿Cómo confía mi navegador en un sitio?** Valida la cadena de firmas hasta una raíz de su almacén de confianza, comprueba validez, SAN y revocación.

 **¿Qué es el certificate pinning?** Fijar una clave/certificado esperado en la app para no depender solo de la PKI; reduce el riesgo de CA comprometida.

Referencias

- IETF RFC 5280 (X.509) — <https://www.rfc-editor.org/rfc/rfc5280>
- Wong, *Real-World Cryptography*, cap. 9.
- RFC 6962 (Certificate Transparency) — <https://www.rfc-editor.org/rfc/rfc6962>
- Let's Encrypt / ACME (RFC 8555) — <https://letsencrypt.org/how-it-works/>

Siguiente clase

Clase 056 - TLS/SSL en profundidad